Find Successor

AlgoExpert · Medium · Binary Trees

The problem

- Nodes have parent pointers; return the in-order successor of `node` (or null).

Intuition

- The in-order successor is the leftmost node of the right subtree, or else the first ancestor reached from a left child.

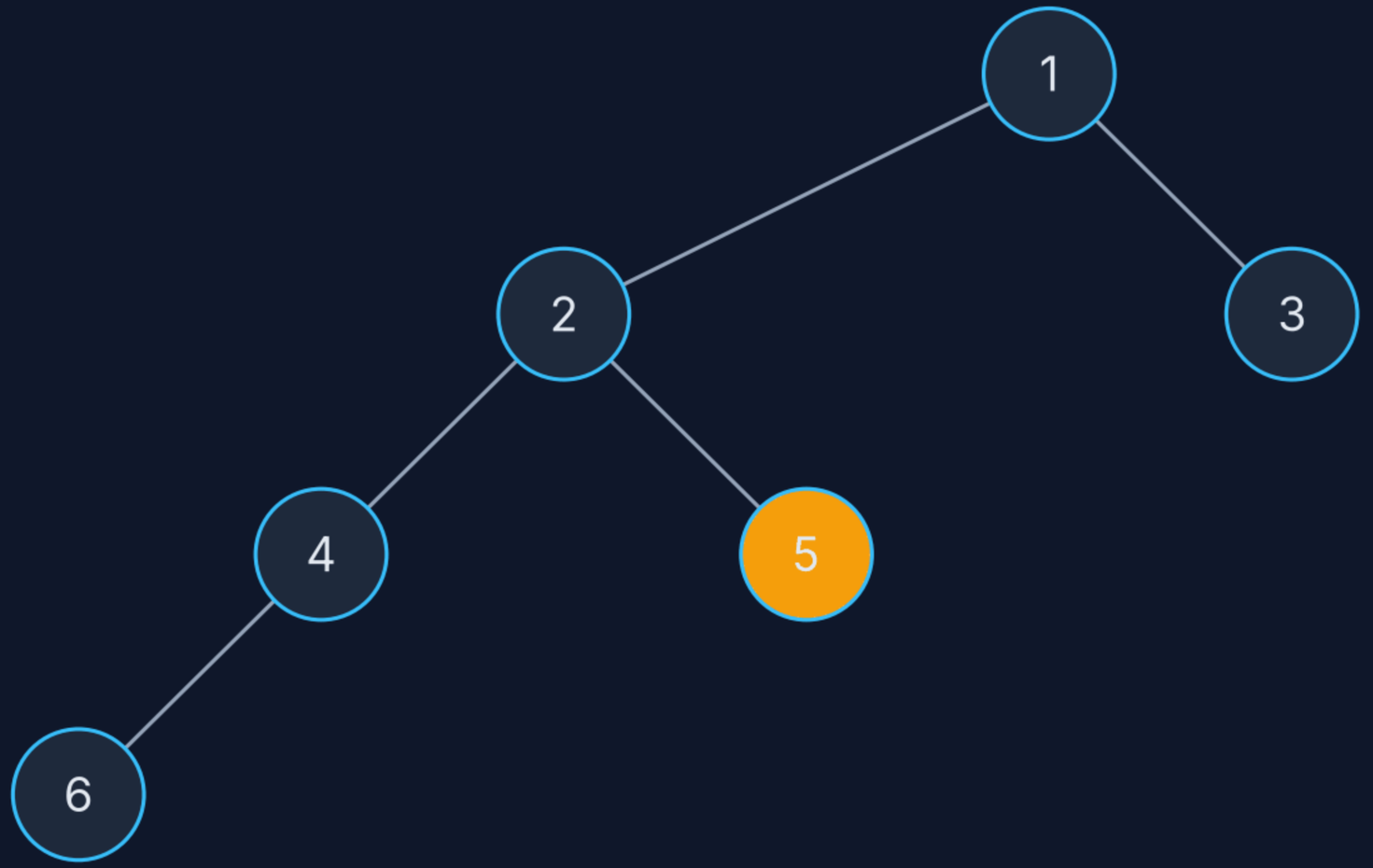
Brute force vs optimal

- In-order traversal into a list, find node, take next → O(n)
- Parent pointers: two cases → O(h) time, O(1) space
- No full traversal needed

Approach

- 1. If there's a right subtree, go right then all the way left
- 2. Otherwise climb parents until we come from a left child

Worked example

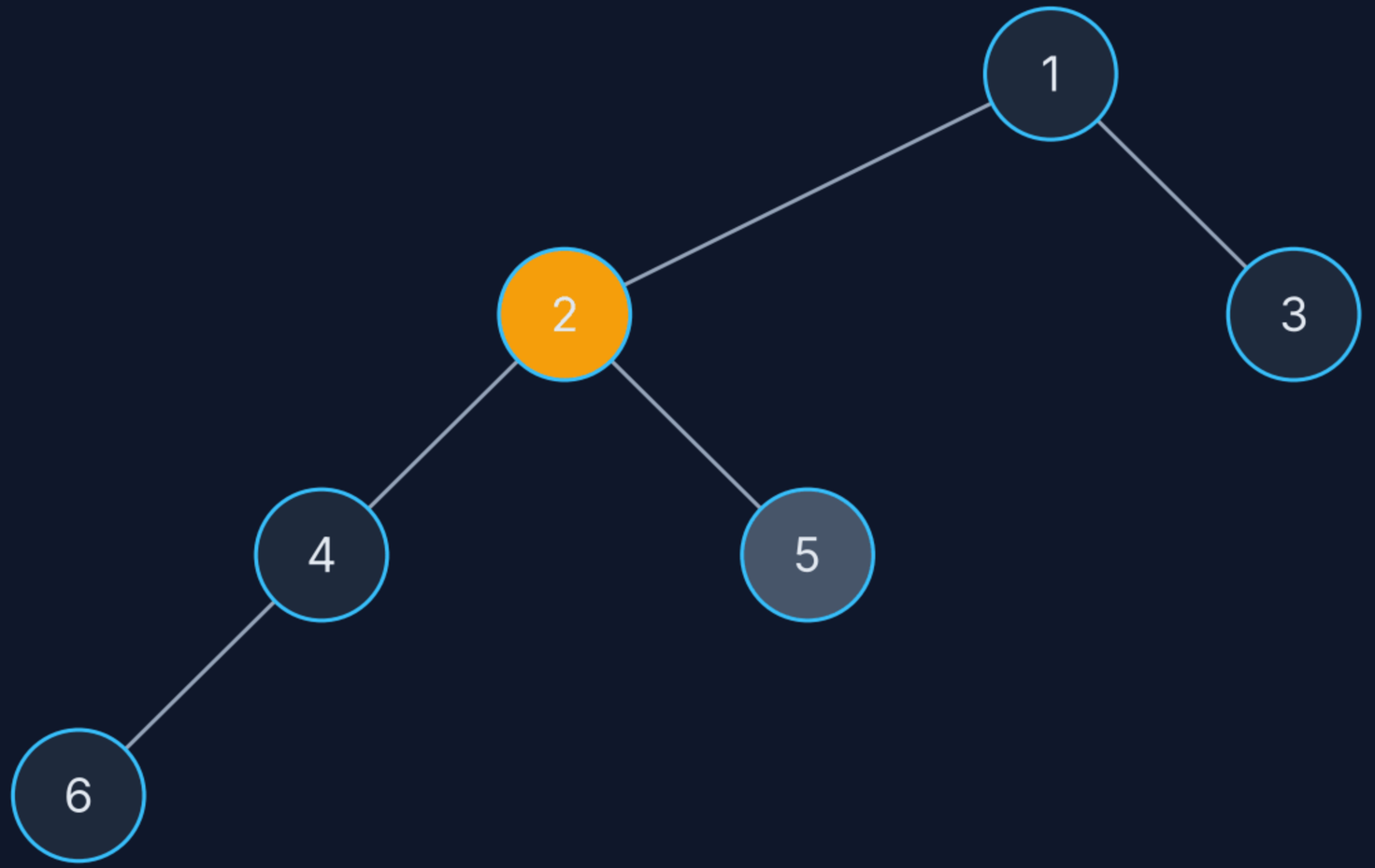


in-order:

6 4 2 5 1 3

Successor of 5: no right child

Worked example

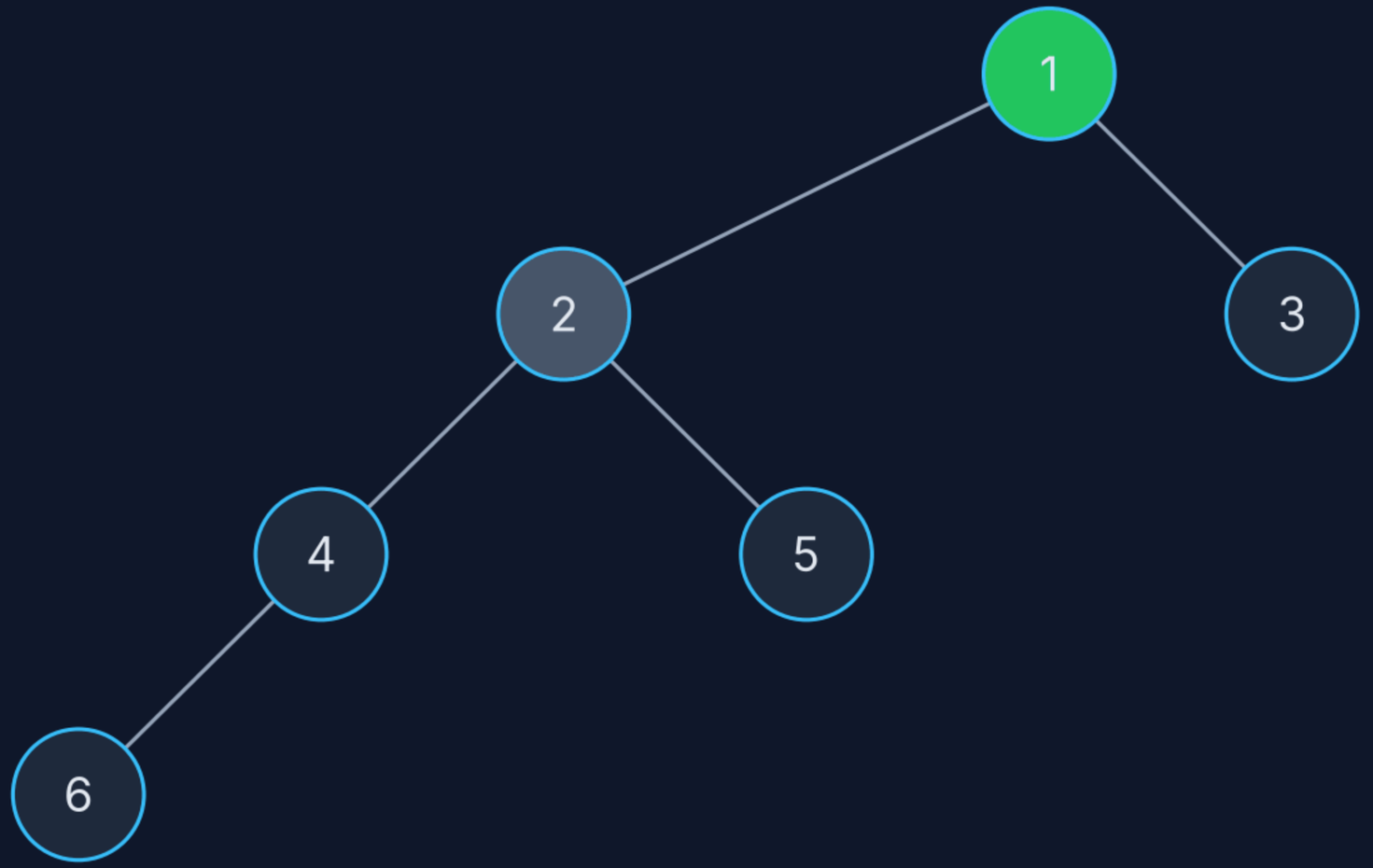


in-order:

6 4 2 5 1 3

5 is 2's RIGHT child → keep climbing

Worked example

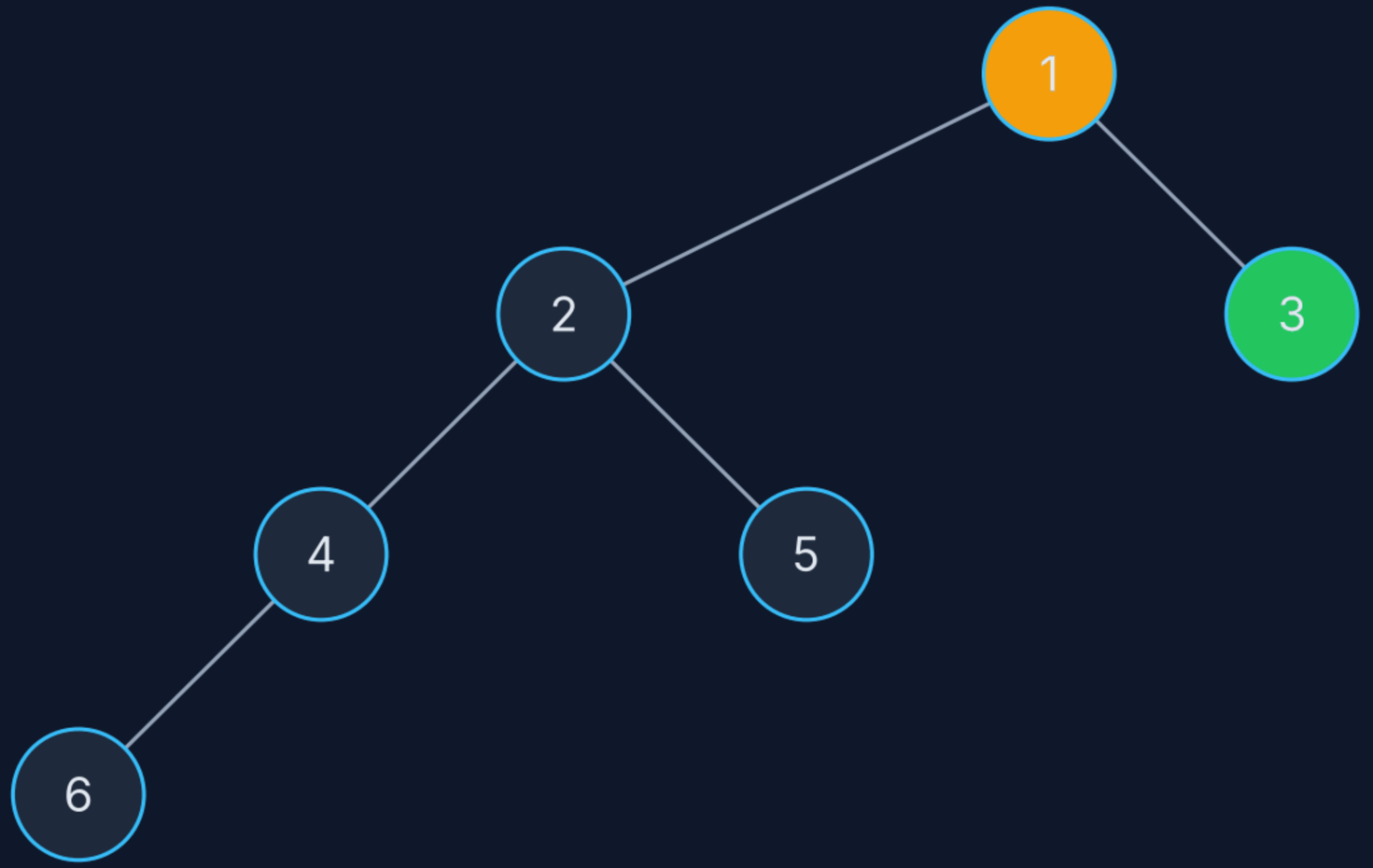


in-order:

6 4 2 5 1 3

2 is 1's LEFT child → successor is 1

Worked example



in-order:

6 4 2 5 1 3

Successor of 1: leftmost of right subtree → 3

Java solution

```java
public class FindSuccessor {
    public static BinaryTree findSuccessor(BinaryTree tree, BinaryTree node) {
        if (node.right != null) {
            BinaryTree c = node.right;
            while (c.left != null) c = c.left;
            return c;
        }
        BinaryTree c = node;
        while (c.parent != null && c.parent.right == c) c = c.parent;
        return c.parent;
    }
}
```

Complexity

- Time: O(h)
- Space: O(1)

Edge cases & pitfalls

- Last node in order → null
- Node with right subtree → leftmost of it
- Climb while we're a right child

Interview tips & follow-ups

- Follow-up: predecessor (mirror image)
- Follow-up: without parent pointers in a BST

Recap

- The in-order successor is the leftmost node of the right subtree, or else the first ancestor reached from a left child.
- Complexity: O(h)